AgroOptimize: module interaction

Modules, the calls between them and the data flow from official sources to the browser

React Frontend (frontend/src)

Views: Dashboard · Run Optimization · Analytics · Comparison · History · Reports · District Intelligence · Scenario Simulation · Official Data
services/api.js: the single place that calls the backend (fetch, same origin in production)

HTTP / JSON

FastAPI REST API (app/main.py)

POST /api/farm-plans · GET /api/results/{id} · /api/results/{id}/explanation · /api/results/{id}/topsis · /api/region-profile
GET /api/dashboard · /api/plans · /api/comparison · /api/compare-fuzzy · /api/district-intelligence · /api/history (per district)
POST /api/simulations/run · GET /api/datasets · /api/soil-master · /api/fuzzy/explain · /api/fuzzy/audit · /health (+ 501 if no official data)

Services

Plan service

Runs the pipeline per request:
preprocess → fuzzy → NSGA-II → TOPSIS

app/services/plan_service.py

Explainability engine

Reasons, risks, confidence, fuzzy rules, TOPSIS, per-crop evidence

explanation_service.py · crop_evidence.py

TOPSIS service

Re-rank saved fronts, profiles, custom weights, comparisons

app/services/topsis_service.py

Dashboard / District / Simulation

KPI cards, Pareto plans, A/B/C, district profile, what-if scenarios

dashboard_ · district_ · simulation_service.py

Results store

Saved runs and simulations (JSON + CSV + convergence)

app/services/results_store.py → results/

stage 1 → 2 → 3 → 4

Algorithm modules

Preprocessing

District + season profile: rainfall vs IMD normal, reservoir %, soil index; crop filtering with reasons

app/data/preprocessing.py

Fuzzy engine

Mamdani FIS (scikit-fuzzy), model 'official' calibrated on the data → yield confidence, irrigation risk

app/fuzzy/engine.py · calibration.py

NSGA-II engine

pymoo NSGA-II, 4 objectives, single-point crossover, Gaussian mutation, quality metrics (HV, IGD+)

app/optimization/

TOPSIS ranking

Criteria, profiles, closeness, per-criterion contributions, plan-vs-plan explanation

app/decision/

reads

calibration

Dataset service (official data access)

app/data/loader.py + official_view.py: crop, land, rainfall and water frames for the pipeline (cached per process)
app/datasets/service.py: SQL queries with TTL cache, provenance; soil_master · agronomy_master · conversion_loader
No CSV fallback: a missing dataset raises OfficialDataUnavailable → HTTP 501 for that feature

SQL

SQLite database official_v1.db

16 tables via SQLAlchemy (AGROOPTIMIZE_DB_URL)
dataset_registry (sha256, rows, report year),
ingestion_log, district, crop_stat, msp, reservoir …

writes

Build pipeline (once, and at Docker build)

scripts/preprocess_all.py → build_database():
per dataset validate → clean → import;
validation_report.json

read raw files

Official government datasets (data/official/raw, report year 2024-25)

TN Season & Crop Report (crops, state series, land use, irrigation, IMD rainfall) · groundwater levels · WRD daily reservoir levels
CACP MSP and cost of cultivation · TNAU Crop Production Guide doses · Soil Health Card 2024-25 · agronomy / soil masters, conversion table

Interface and services · Decision algorithms · Official data layer

The explanation, dashboard, district and simulation services also call preprocessing, the fuzzy engine, NSGA-II and ranking directly (imports in app/services).

Source: imports and routes in app/ and frontend/src/services/api.js (branch feature/official-data-migration)